

Frontend — Arquitetura e Módulos

Stack e padrões gerais

- **React + Vite**, sem client-side router — a navegação entre módulos é feita trocando um estado (`activeModule` em `AppInner` , dentro de `App.jsx`) que seleciona qual componente renderizar via `RENDER_MAP` . Não há URLs distintas por módulo.
- **Estilização**: quase 100% via objetos de estilo inline (`style={{ ... }}`) diretamente nos componentes — é o padrão dominante em todo o código. O Tailwind está disponível no projeto (classes aparecem em alguns arquivos mais novos, como `NetMonitor/frontend`), mas dentro do Dashboard propriamente dito o uso é bem esporádico/legado.
- **Gráficos**: biblioteca **Recharts** em todo o app (`BarChart` , `LineChart` , `ComposedChart` , `PieChart` , `AreaChart` , `ResponsiveContainer` , etc.), importada uma vez no topo de `App.jsx` e reexportada implicitamente para os componentes locais que os usam.
- **Busca de dados**: hook customizado `useFetch(path, deps={}, intervalMs=0)` (definido em `App.jsx` , e uma cópia local equivalente em `PacientesDB.jsx`). Ele monta a query string a partir do objeto `deps` (filtrando `null` / `""`), refaz o fetch sempre que `path` ou `JSON.stringify(deps)` mudam, e opcionalmente reconsulta em intervalo fixo (usado em painéis "tempo real"). Retorna `{ data, loading, error }` .
- **URL base da API**: `const API = \ ${window.location.protocol}//${window.location.host}` — sempre relativo ao host atual, então funciona tanto no IP interno (https://192.168.1.40:31000) quanto por túnel externo, sem precisar reconfigurar nada.`
- **PWA**: `index.html` registra um Service Worker (`/sw.js`) e referencia `manifest.json` (nome "Censo · Gestão Clínica", cor tema `#8B1A1A` , ícones em vários tamanhos incluindo variantes "maskable" para Android).
- **Branding**: logo da Clínica Censo (base64 embutido) no canto esquerdo da topbar, e logo do ICDS (`/icds_logo.png`) no canto direito, ambos dentro de caixas brancas arredondadas sobre o gradiente vermelho (`#8B1A1A` → `#6B1414`) da topbar.

Autenticação (`Login.jsx`)

- `AuthProvider` guarda o usuário logado em `sessionStorage` (chave `censo_user`), expõe `login()` , `logout()` e `podeVer(modulo)` (cheça se o usuário é admin ou se `modulo` está na lista `user.modulos`) via `AuthContext` / `useAuth()` .
- Componente `Login` (default export): formulário de login que chama `POST /api/auth/login` .
- `AdminPermissoes` (export nomeado, renderizado pelo módulo "admin"): tela de gestão de permissões — busca usuários (`/api/auth/usuarios`), lista módulos (`/api/auth/modulos`), lista usuários por grupo (`/api/auth/usuarios-por-grupo`) e salva/atualiza permissões (`/api/auth/permissoes`).

Navegação e mapeamento de módulos

`NAV` (array em `App.jsx`) define os itens do menu principal — cada um com `id` , `label` , `icon` , `color` e `desc` :

id	Label	Descrição
<code>home</code>	Home	Página inicial/resumo geral
<code>contratos</code>	Contratos	Gestão de contratos (redireciona a um app externo)

id	Label	Descrição
faturamento	Faturamento	Guias pendentes de faturamento
clinica	Clínica	Assistencial · Ocupacional · Serviços · Agenda
atendimento	Atendimento	Fila do médico · Prontuário
laboratorio	Laboratório	Exames · Diagnóstico · Ocupacional
recepcao	Recepção	Métricas por recepcionista
producao	Produção Mensal	Meta e provisionamento mensal
pacientesdb	Pacientes	Base · logradouros · ranking · aniversários
estoque	Estoque	Posição, giro e validade
painel_tv	Painel TV	Tempo real, para telão
admin	Permissões	Gerenciar acessos

`RENDER_MAP` liga cada `id` ao componente real, repassando o período efetivo (`p` , vindo do seletor de período global da topbar) quando o módulo precisa dele:

```
home:      (p) => <Home periodoGlobal={p}/>,
admin:     () => <AdminPermissoes/>,
contratos: () => <ModuloContratos/>,
faturamento: () => <Faturamento/>,
clinica:   (p) => <SecaoClinica periodo={p}/>,
atendimento: () => <Atendimento/>,
recepcao:  (p) => <Recepcao periodo={p}/>,
pacientesdb: (p) => <PacientesDB periodo={p}/>,
producao:  (p) => <SecaoProducaoModulo periodoEfetivo={p}/>,
laboratorio: (p) => <SecaoModuloLaboratorio periodo={p}/>,
estoque:   (p) => <SecaoEstoque periodo={p}/>,
painel_tv: () => <PainelTV/>,
```

Seletor de período global

A topbar tem um seletor de período (`PERIODS` : Hoje / Mês atual / Ano atual) mais um seletor de intervalo personalizado (`SeletorPeriodo` /date picker com `MiniCal` , dois calendários lado a lado para escolher data início/fim). O valor escolhido é propagado como prop pros módulos que aceitam período.

Componentes reutilizáveis principais (definidos em `App.jsx`)

- `Card` — container branco padrão com cabeçalho opcional (título, subtítulo, ação, barra de cor lateral `accent`).
- `KPI` — card de indicador único com gradiente de cor, valor grande, selo de variação (`sub` , `deltaUp`).
- `ModuloCard` — variante de KPI com ícone circular colorido, usada nos módulos "clássicos" (Assistencial, Ocupacional, Estoque, etc.).
- `ModuleHero` — cabeçalho hero em gradiente com estatísticas em cards translúcidos, usado no topo de vários módulos.
- `Skeleton` — placeholder de carregamento (shimmer animado).
- `Err` — banner de erro padrão.

- **GraficoComparativoAnual** — gráfico de linha comparando anos diferentes lado a lado (usa **SeletorAnos**).
- **MetaModulo** / **PainelMetas** / **useMetas** — mecanismo de meta mensal por módulo, persistido via **/api/metas** .

Módulos — detalhamento

Produção Mensal (**SecaoProducaoModulo** , dentro de **App.jsx**)

Dividido em abas (**ABAS_PRODUCAO**): - **Visão Geral** (**SecaoProducaoMensal**) — produção diária ocupacional × assistencial vs. meta, tabela mensal, ranking de profissionais (**ProducaoProfissionais**), gráfico comparativo anual (**/api/financeiro/producao-mensal** , **/api/financeiro/recordes**). - 🏧 **Fluxo de Caixa** (**SecaoFluxoCaixa**) — KPIs de entradas/saídas/saldo/saldo projetado, gráfico diário (**ComposedChart**), projeção de 30 dias, despesas por categoria, top fornecedores, comparativo mensal faturado×recebido×pago. Consome toda a família **/api/financeiro/fluxo-caixa/*** e **/api/financeiro/comparativo** . Inclui o modal **NovaDespesaModal** — formulário de lançamento de despesa nova (fornecedor com autocomplete, tipo pessoa, centro de custo, valor/parcelas) que grava via **POST /api/financeiro/despesas** .

Clínica (**SecaoClinica**)

Abas (**ABAS_CLINICA**): - **Assistencial** (**SecaoModuloAssistencial**) — **/api/modulo/assistencial/resumo** . - **Ocupacional** (**SecaoModuloOcupacional**) — **/api/modulo/ocupacional/resumo** , inclui **CardTopEmpresas** . - **Serviços Espec.** (**SecaoModuloServicos**) — **/api/modulo/servicos/resumo** . - **Agendamentos** (**SecaoModuloAgendamentos**) — **/api/modulo/agendamentos/resumo** e **-resumo-hoje** (atualização a cada 30s), tabela **TabelaMedicosAgenda** . - 📅 **Agenda do Médico** (**PainelAgendaMedico**) — seleção de médico com avatar, visão diária (timeline) e mensal, resumo do mês (faturamento, absenteísmo, cancelados). Usa **/api/agenda/medicos** , **/api/agenda/dia** , **/api/agenda/mensal** . - 🕒 **Valor/Hora** (**PainelValorHora**) — duas tabelas (**TabelaValorHora**): valor gerado por hora por **médico** (**/api/agenda/medicos/valor-hora** , atribuição pelo executor real via SMM_MED) e por **consultório** (**/api/agenda/consultorios/valor-hora**) — a tabela por consultório hoje mostra só uma linha, pois a agenda usa um registro de sala praticamente único.

Faturamento (**Faturamento.jsx** , componente próprio fora de **App.jsx**)

- Duas abas internas: **lista** (guias pendentes, CRUD completo — criar/editar/excluir/mudar status, com modal de motivo obrigatório ao cancelar) e **dashboards** (**DashboardsFaturamento**).
- Hero fixo **FaixaResumoGeral** mostra **somente o valor total de guias pendentes** de todo o período (ajustado a pedido do usuário — antes somava também Entregue).
- **CardsResumo** — breakdown por status (Pendente/Entregue/Cancelada) + card de "pendentes há 30+ dias".
- Endpoints: **/api/faturamento/guias** (GET/POST/PUT/DELETE), **/api/faturamento/resumo** , **/api/faturamento/dashboard** .

Recepção (**Recepcao.jsx**)

Duas abas (**abaRecep**): - **Visão Geral** — filtro por recepção (Consultórios/Diagnóstico/Ocupacional/Censo Imagem), KPIs, metas de recepção (**/api/recepcao/metas**), ranking de recepcionistas (**/api/recepcao/ranking** , ordenável, com breakdown de convênios ao expandir uma linha via **/api/recepcao/convenios**), gráfico de evolução diária por turno (**/api/recepcao/evolucao**), **GraficoMediaPorHorario** (**/api/recepcao/media-por-horario**). - ⌚ **Pontualidade** (**PainelPontualidade**) — seletor de recepcionista (**/api/recepcao/usuarios**) + período, botão "Gerar Relatório" que busca **/api/recepcao/pontualidade** (compara horário de login — **GR_SES** — com o horário de criação da primeira OS do dia — **OSM_DTHR**), e botão "Baixar PDF" que baixa o mesmo relatório via **/api/recepcao/pontualidade/pdf** (gerado no backend com Chrome headless).

Pacientes (**PacientesDB.jsx**)

Tem seu próprio **useFetch** local (mesmo padrão). Seções: resumo geral, distribuição por bairro (**/api/pacientesdb/por-bairro**), faixa etária e sexo (**/api/pacientes/faixa-etaria** , **/api/pacientes/por-sexo**), crescimento da base

(`/api/pacientesdb/crescimento-base`), retorno vs. novos pacientes (`/api/pacientesdb/retorno-vs-novos`), ranking de top atendimentos (`/api/pacientes/top-atendimentos`), aniversariantes do mês (`/api/pacientes/aniversariantes`).

Laboratório (`SecaoModuloLaboratorio` , em `App.jsx`)

Consome `/api/modulo/laboratorio/resumo` , `/bancadas` , `/por-recepcao` , `/tempo-coleta` , `/producao-por-profissional` ; inclui `DashboardRecoleta` (`/api/laboratorio/recoleta`), `TabelaMedicosLab` , `TabelaTopEmpresas` .

Estoque (`SecaoEstoque` , em `App.jsx`)

Módulo extenso com muitas sub-visões: resumo (`/api/estoque/resumo`), posição atual com filtro por curva ABC e busca (`/api/estoque/posicao` , `TabelaPosicaoEstoque`), giro (`/api/estoque/giro`), lotes a vencer (`/api/estoque/lotos-vencimento`), movimentações (`/api/estoque/movimentacoes`), curva ABC (`/api/estoque/curva-abc`), movimentação por dia/grupo/setor, visões sintética e analítica sob demanda (`skip` condicional no `useFetch` pra não buscar até a aba ser aberta).

Atendimento (`Atendimento.jsx`)

Tela de "fila do médico" / prontuário (rotulada como "teste" no NAV). Busca fila por médico (`/api/atendimento/fila`), histórico do paciente (`/api/atendimento/historico`), busca de paciente/CID/procedimento/convênio via campos de autocomplete próprios, dados clínicos (`/api/atendimento/clinica`), template de formulário por serviço (`/api/atendimento/template`), e grava o atendimento via `POST /api/atendimento/salvar` (que grava em `smart_html`, homologação, não em produção, até validação).

Contratos (`ModuloContratos.jsx`)

O mais simples de todos — não chama nenhuma API própria. É uma tela com um card e um botão que abre, em nova aba, um app externo hospedado no Netlify (<https://frolicking-pavlova-d2b8af.netlify.app/>).

Home (`Home.jsx`)

Página inicial: resumo geral (`/api/home/resumo?periodo=&setor=`) e briefing executivo gerado por IA (`POST /api/home/briefing` , mesmo padrão do `BriefingCard` reutilizado em outros módulos como Recepção).

Painel TV (`PainelTV.jsx`)

Painel "modo telão" pra exibição em tempo real (atualização automática via `usePainelFetch` , polling a cada ~30s). Mostra faturamento do dia vs. meta diária (`/api/metast` , `/api/painel/resumo-hoje`), barra de progresso, e uma feature distintiva: **sistema de som progressivo por marco de meta** (`tocarSomMarco`) — a cada 10% da meta batida toca um riff eletrônico sintetizado via Web Audio API (ondas dente-de-serra, acelera e ganha um "kick" de bateria a partir de 60%); ao bater 100% da meta, em vez do riff sintetizado, toca um arquivo de áudio real (`/som_meta_batida.mp3` — "Eu vou tomar um tacacá", Joelma) e ativa uma animação de celebração na tela (fundo verde pulsante, `celebrando`).

Outros arquivos auxiliares

- `BriefingCard.jsx` — card reutilizável de briefing executivo gerado por IA, com cache local (`cacheKey`) pra não regenerar a cada render.
- `MeusResultados.jsx` , `GraficoProducaoRecepcao.jsx` , `MapaParauapebas.jsx` , `ServicosPorSexo.jsx` — componentes de suporte/visualizações específicas usados pontualmente em outros módulos.